



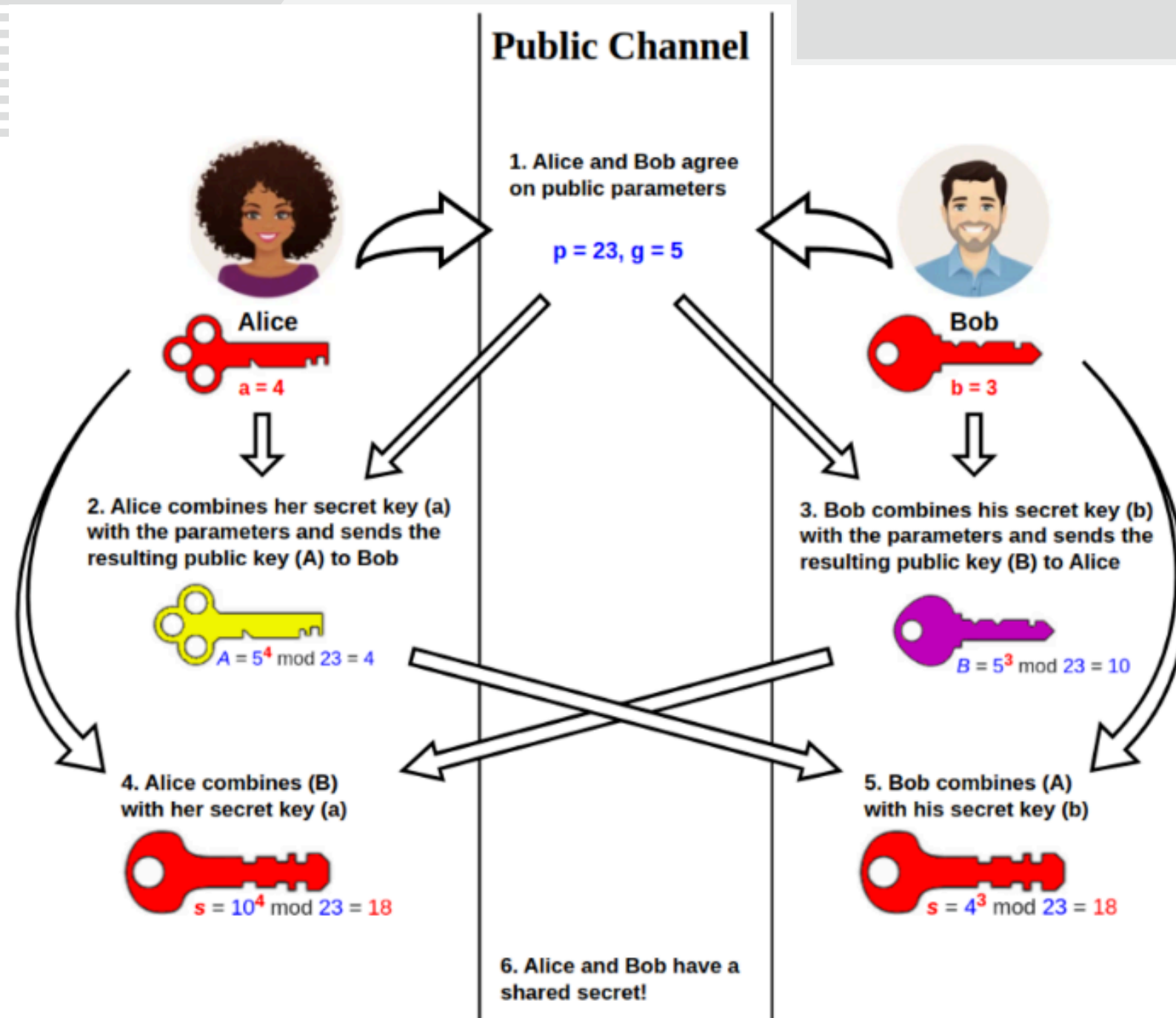
מעבדת תכן רשתות פרוייקט מסכם צ'אט מוצפן



שיר משה

עידו בן בעש"ט

DIFFIE HELLMAN



- השרת מגריל פרמטרים p, g, a ושולח ללקוח:
 $g, p, g^a \bmod p$

- הלקוח מגריל פרמטר b ושולח לשרת:
 $g^b \bmod p$

- כל אחד מהם מחשב את מפתח ההצפנה המשותף שלהם: $g^{ab} \bmod p$
(מעלים את המפתח שקיבלו מהאחר בחזקת הפרמטר)

IMPLEMENTATION

PHASE 1

החלפת מפתחות בין
השרת ללקוח וקבלת
מפתח משותף ייחודי
ללקוח זה

PHASE 2

הסרבר שולח את
מפתח המולטיקאסט
מוצפן ע"י המפתח
של הקליינט

PHASE 3

הקליינט שולח
הודעות מוצפנות (ע"י
מפתח ייחודי) לסרבר
ומקשיב להודעות
מוצפנות של קבוצת
המולטיקאסט

PHASE 4

הסרבר מפענח
הודעות מהלקוחות,
מצפין במפתח
המולטיקאסט ומפיץ
לכל הקבוצה



TOPOLOGY

התבססנו על טופולוגיה מבוססת OSPF עם PIM

TCP Socket - סוקט יחיד לכל לקוח עבור הקמת קשר ראשוני, החלפת מפתחות בין השרת ללקוח, העברת מפתח, ושליחת הודעה להצפנה. בנוסף שימש גם לשליחת הודעות keep alive.

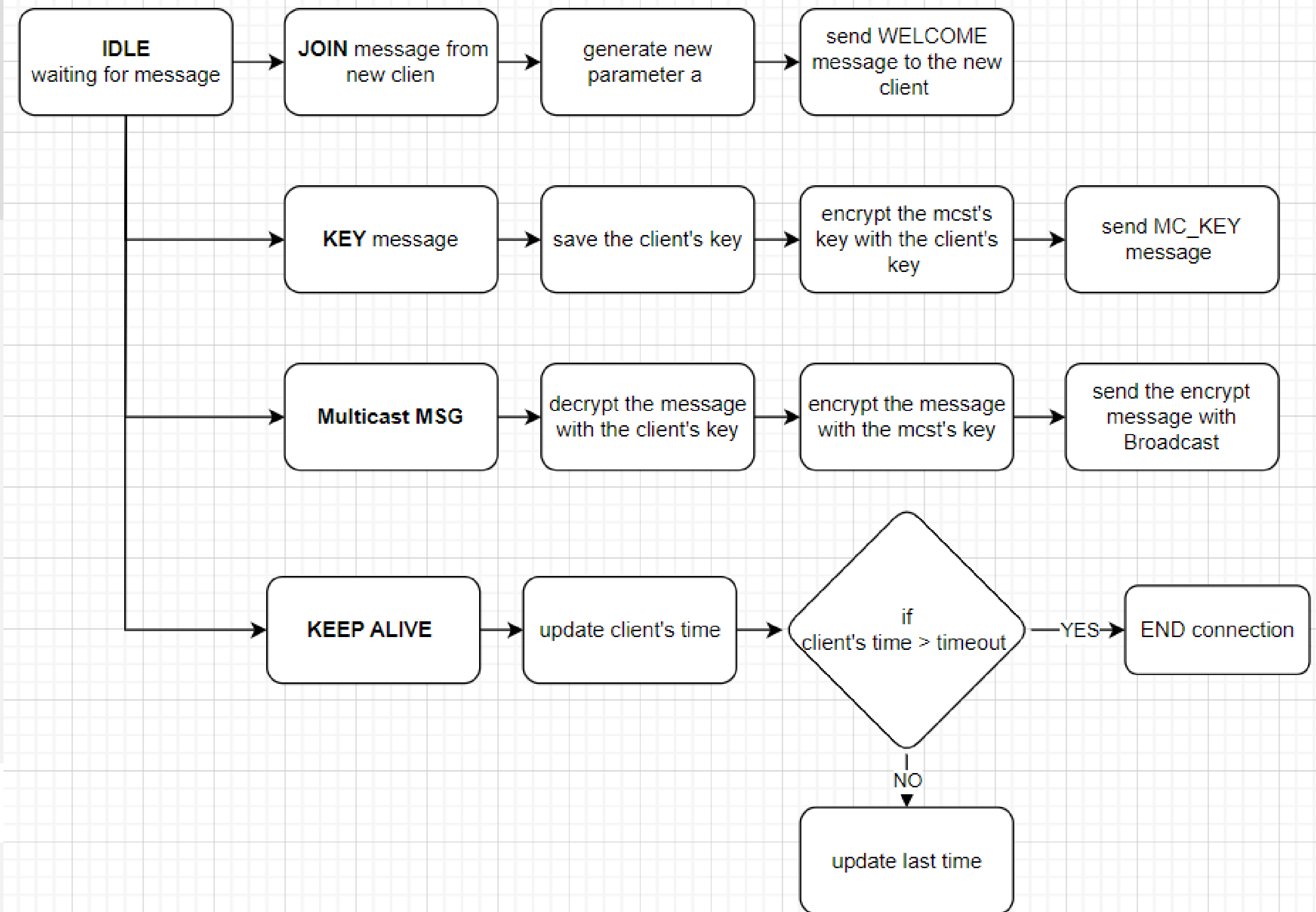
UDP Socket - סוקט אחד שמנוהל ע"י הסרבר עבור הודעות ה-Broadcast של כלל הקבוצה, שימוש עיקרי עבור הפצת ההודעות המוצפנות לכלל הקבוצה.



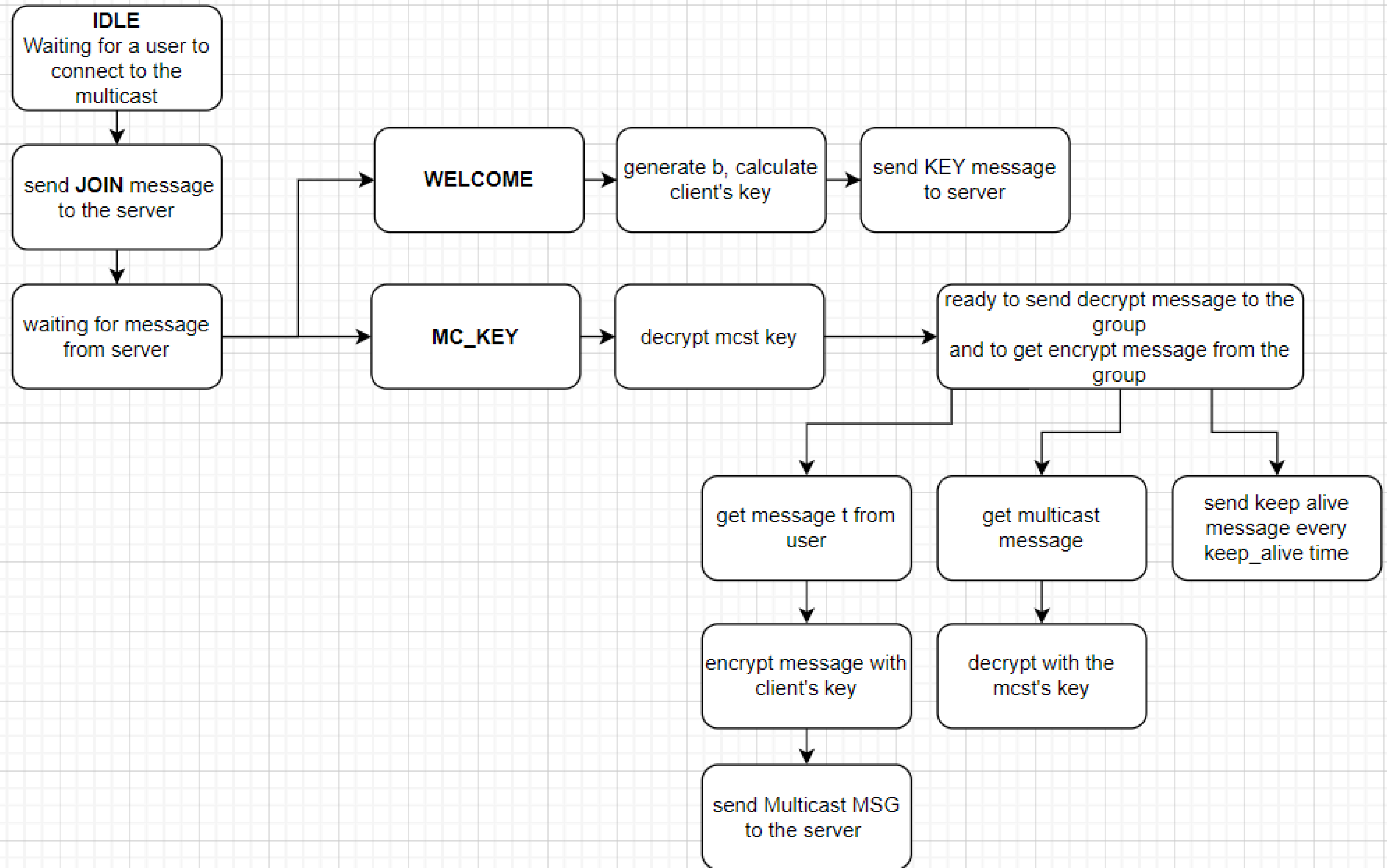
MESSAGE TYPE

- הודעת WELCOME - הסרבר שולח לקליינט את הפרמטרים הבסיסיים כמו g , p , ואת $g^a \bmod p$
- הודעת MC_KEY - הסרבר שולח לקליינט את המפתח של הקבוצה מוצפן באמצעות המפתח הייחודי של הקליינט
- הודעת MULTICAS - הסרבר מפיץ לכל הקבוצה הודעת מולטיקאסט של הדתא המוצפן ע"י המפתח הקבוצתי
- הודעת KEEP ALIVE - כל קליינט שולח אחת לזמן מה הודעת keep alive (אם חרג מה timeout הקשר עימו מנותק)
- הודעת JOIN - כאשר קליינט מעוניין להתחבר לשרת לשירות הצפנת המולטיקאסט
- הודעת KEY - הקליינט שולח את המפתח שלו $(g^b \bmod p)$
- הודעת Multicast MSG - הקליינט שולח הודעה מוצפנת במפתח האישי לסרבר

FSM SERVER



FSM CLIENT



TEST

כשיש timeout:

```
root@pc2:~/Lab_DH_project# gcc client2708_1.c -o clientlast -lm -pthread
root@pc2:~/Lab_DH_project# ./clientlast 192.13.3.1
Hello Welcome to Our Safe Chat Group
Please Enter Your host ID: (expected value 1-10): 1
Connecting to server at IP: 192.13.3.1, Port: 8080
before create a udp thread
--> Send: Message Type:(0)Client joining, my id: 1
in KEEP ALIVE thread
Join successefully to UDP socket
<-- Recieve: Message Type:(1)WELCOME, server key: 5, p: 7, g: 3
Generated Integer b: 6
client key: 1, server key: 5, shared key:1
<-- Recieve: Message Type:(3)MC_KEY: 4
Decrypted multicast key: 5
Client successfully switched keys
If you want to send an encrypted message to the group press y: y
Enter your message: helo

<-- Received: UDP message: m`ij
Decrypt UDP message: helo
If you want to send an encrypted message to the group press y: close connection
oot@pc2:~/Lab_DH_project#
```

```
Generate DH parameters:
g = 3, p=7, multicast key=5
OK: bind SUCCESS
Socket is bound to:
  addr = 0
  port = 38244
Server listening on IP: 192.13.3.1, Port: 8080
in UDP function
Client connected
Generated Integer a: 5
<-- Receive: Message Type:(0)Client joining, my id: 1
Type msg: 0
--> Send: Message Type:(1)WELCOME, server key: 5, p: 7, g: 3
<-- Receive: Message Type:(2)KEY: 1
Type msg: 2
--> Send: Message Type:(3)MC_KEY: 4
<-- Receive: Message Type:(4): The Encrypt Message is: idmn

Type msg: 4
Data is: idmn
IN DECRYPT: helo
return from decrypt function
decrypt_msg: helo
get SIGNAL from tcp thread
encrypt MULTICAST msg: m`ij
--> Send: m`ij
Client 0 timed out
--> sent: Message Type:(9) GoodBye!
```